



PROGRAMMING WITH PYTHON

Lekha A

Computer Applications

PROGRAMMING WITH PYTHON

Data Ingestion and Multiprocessing

Web Scraping using Python

Lekha A

Computer Applications



PROGRAMMING WITH PYTHON

Web Scraping

- Web scraping refers to the practice of utilizing a program or algorithm to gather and process substantial quantities of data from the internet.
- The `urllib.request` module serves the purpose of opening URLs.
- The BeautifulSoup package is utilized for extracting data from HTML files.
- The name of the BeautifulSoup library is `bs4`, which represents BeautifulSoup, version 4.



PROGRAMMING WITH PYTHON

Web Scraping

```
pip install beautifulsoup4
```

Defaulting to user installation because normal site-packages is not writeable

Requirement already satisfied: beautifulsoup4 in c:\users\dell\appdata\local\packages\pythonsoftwarefoundation.python.3.13_qbz5n2kfra8p0\localcache\local-packages\python313\site-packages (4.13.4)

Requirement already satisfied: soupsieve>1.2 in c:\users\dell\appdata\local\packages\pythonsoftwarefoundation.python.3.13_qbz5n2kfra8p0\localcache\local-packages\python313\site-packages (from beautifulsoup4) (2.6)

Requirement already satisfied: typing-extensions>=4.0.0 in c:\users\dell\appdata\local\packages\pythonsoftwarefoundation.python.3.13_qbz5n2kfra8p0\localcache\local-packages\python313\site-packages (from beautifulsoup4) (4.13.2)

Note: you may need to restart the kernel to use updated packages.

```
from urllib.request import urlopen  
from bs4 import BeautifulSoup
```



PROGRAMMING WITH PYTHON

Web Scraping

- Specify the URL containing the dataset and pass it to `urlopen()` to get the html of the page.

- ```
url = "https://tatamumbaimarathon.procam.in/results/race-results"
'''a variable url is defined and assigned the value
of the URL of the webpage to be retrieved.'''
html = urlopen(url)
'''urlopen() function from the urllib.request module is used to open the URL
and retrieve its content. The content is returned as a file-like object
which can be read and processed further.'''
```

```
type(html)
```

```
http.client.HTTPResponse
```



# PROGRAMMING WITH PYTHON

## Web Scraping

---

- Construct a BeautifulSoup object using the HTML.
- This is achieved by supplying the HTML to the BeautifulSoup() function.
- The BeautifulSoup library is utilized to interpret the HTML, meaning it takes the unprocessed HTML text and converts it into Python objects.
- The second parameter, lxml, refers to the HTML parser.



# PROGRAMMING WITH PYTHON

## Web Scraping

- ```
soup = BeautifulSoup(html, 'lxml')  
'''returns a soup object, which is an object representation of the HTML code'''  
type(soup)
```

bs4.BeautifulSoup

- Extract the title of a webpage

- ```
title = soup.title
print(title)
```

```
<title>TMM | Race Results</title>
```



# PROGRAMMING WITH PYTHON

## Web Scraping

- Get the text of the webpage and quickly print it out to check if it is what was expected.

```
text = soup.get_text()
'''The get_text() method is called on the soup object to extract the
text from the webpage. The extracted text is then assigned to the variable text.'''
print(soup.text)
```

```
TMM | Race ResultsTake the Runner Survey! Tata Mumbai Marathon | 18th January 2026 | 37 days 0 hours 54 minutes 48 seconds Take the Runner Survey!Home/
Results/Race ResultsRace Results 20252024202320202019201820172016201520142013201220112010200920082007200620052004Check your Timed Results hereTimed Re
sultsFilterMarathon Overall Elite - MenMarathon Overall Elite - WomenMarathon Indian Elite - MenMarathon Indian Elite - WomenPolice CupMarathon - Age C
ategoryHalf Marathon - Age CategoryOpen 10K - Age CategoryRANKBIBNAMENATIONALITYFINISH TIME110BERHANE TESFAYEritrea02:11:4427MERHAWI KESETEEritrea02:1
1:50313TESFAYE DEMEKEEthiopia02:11:5644PHILEMON RONOKenya02:12:0956ABDI FUFAEthiopia02:12:4861LEMI BERHANUEthiopia02:14:54726ANISH THAPAIIndia02:17:2382
2MAN SINGHIndia02:17:3798KEMAL HUSENEthiopia02:17:37102ASRAR HIYRDENEthiopia02:18:311124GOPI THONAKALIndia02:19:591227KALIDAS HIRAVEIndia02:20:281321SR
INU BUGATHAIIndia02:20:431425PRADEEP SINGH CHAUDHARYIndia02:22:241567BJORN KOREMANNetherlands02:25:011637DEV SINGH DAGARIndia02:25:591735LATE CHANGDEVIn
dia02:26:241838PRATHAMESH PARAMKARIndia02:28:411936ANANT GAONKARIndia02:29:142033YOGESH KUMAR SHARMAIndia02:29:47
```





- and selecting "Inspect."



**TATA MUMBAI MARATHON**  
TATA CONSUMER SERVICES  
18 January, 2024

[REGISTER / SIGN IN](#)



2025

▼

## Check your Timed Results here

[TIMED RESULTS](#)

Marathon  
Overall Elite – Men

Marathon  
Overall Elite – Women

Marathon  
Indian Elite – Men

Marathon  
Indian Elite – Women

RANK	BIB	NAME	NATIONALITY	FINISH TIME
1	10	BERHANE TESFAY	Eritrea	02:11:44
2	7	MERHAWI KESETE	Eritrea	02:11:50
3	13	TESFAYE DEMEKE	Ethiopia	02:11:56
4	4	PHILEMON RONO	Kenya	02:12:09
5	6	ABDI FUFA	Ethiopia	02:12:48
6	1	LEMI BERHANU	Ethiopia	02:14:54



Elements

```

<!DOCTYPE html>
<html lang="en">
<head>
</head>
<body>
<style id="fonts">
</style>
<div id="_next">
<div class="RunnerSurvey_survey_wrap_pp3t bg_secondary">
<div>
<div>
<div class="Header_main_header_AqR3H">
<div class="Header_bottom_logo_3aM44 next_image">
<main>
<div class="InnerBanner_inner_banner_wrap_gldgu bg_primary">
<div class="innerPageContent">

```

html body div#\_next div main div.innerPageContent

Styles

Filter

show .cls

element.style {

.innerPageContent {

padding-top: 62px;

padding-bottom: 80px;

\* {

box-sizing: border-box;

}

\*, body {

margin: 0;

}

div {

display: block;

unicode-bidi: isolate;

}

Console

AI assistance

What's new X

What's new in DevTools 142

See all new features

See past highlights from Chrome 141



# PROGRAMMING WITH PYTHON

## Web Scraping

---

- Use the `find_all()` method of soup to extract useful html tags within a webpage.
- Examples of useful tags include
  - `< a >` for hyperlinks, `< table >` for tables, `< tr >` for table rows, `< th >` for table headers and `< td >` for table cells.



# PROGRAMMING WITH PYTHON

## Web Scraping

```
soup.find_all('a')
...
```



```
[Take the Runner Survey!
>,
 ,
 Take the Runner Survey!
>,
 ,
 Home,
 Race Results,
 <button class="btn_default">Timed Results</button>,
 <span style="box-sizing:border-box;display:inline-block;overflow:hidden;width:initial;h
eight:initial;background:none;opacity:1;border:0;margin:0;padding:0;position:relative;max-width:100%"><span style="box-sizing:border-box;display:block;
width:initial;height:initial;background:none;opacity:1;border:0;margin:0;padding:0;max-width:100%"><img alt="byTingIcon" data-nimg="intrinsic" decoding="async" sr
c="data:image/gif;base64,R0lGODlhAQABAIAAAAAAAP//yH5BAEAAAAALAAAAABAAEAAAIBRAA7" style="position:absolute;top:0;left:0;bottom:0;right:0;box-sizing:bo
rder-box;padding:0;border:none;margin:auto;display:block;width:0;height:0;min-width:100%;max-width:100%;min-height:100%;max-height:100%"/><noscript><im
g alt="byTingIcon" data-nimg="intrinsic" decoding="async" loading="lazy" src="/_next/static/media/made_by_ting.78029937.svg" srcset="/_next/static/medi
a/made_by_ting.78029937.svg 1x, /_next/static/media/made_by_ting.78029937.svg 2x" style="position:absolute;top:0;left:0;bottom:0;right:0;box-sizing:bor
der-box;padding:0;border:none;margin:auto;display:block;width:0;height:0;min-width:100%;max-width:100%;min-height:100%;max-height:100%"/></noscript></s
pan>]
```



# PROGRAMMING WITH PYTHON

## Web Scraping

- Use a for loop and the get("href") method to extract and print out only hyperlinks.

```
all_links = soup.find_all("a")
for link in all_links:
 print(link.get("href"))
```

```
https://pages.cm.com/12527937000/62046d21-8cae-46d0-bb1e-01a2b25ed798/section/0
/
https://pages.cm.com/12527937000/62046d21-8cae-46d0-bb1e-01a2b25ed798/section/0
None
/
/results/race-results
https://sportstimingsolutions.in/
https://www.ting.in
```

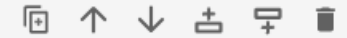


# PROGRAMMING WITH PYTHON

## Web Scraping

- Print out table rows only, pass the tr argument in soup.find\_all().

```
Print the first 10 rows
rows = soup.find_all('tr')
print(rows[:10])
```



```
[<tr><th>RANK</th><th>BIB</th><th>NAME</th><th>NATIONALITY</th><th>FINISH TIME</th></tr>, <tr><td>1</td><td>10</td><td>BERHANE TESFAY</td><td>Eritrea</td><td>02:11:44</td></tr>, <tr><td>2</td><td>7</td><td>MERHAWI KES ETE</td><td>Eritrea</td><td>02:11:50</td></tr>, <tr><td>3</td><td>13</td><td>TESFAYE DEMEKE</td><td>Ethiopia</td><td>02:11:56</td></tr>, <tr><td>4</td><td>4</td><td>PHILEMON RONO</td><td>Kenya</td><td>02:12:09</td></tr>, <tr><td>5</td><td>6</td><td>ABDI FUFA</td><td>Ethiopia</td><td>02:12:48</td></tr>, <tr><td>6</td><td>1</td><td>LEMI BERHANU</td><td>Ethiopia</td><td>02:14:54</td></tr>, <tr><td>7</td><td>26</td><td>ANISH THAPA</td><td>India</td><td>02:17:23</td></tr>, <tr><td>8</td><td>22</td><td>MAN SINGH</td><td>India</td><td>02:17:37</td></tr>, <tr><td>9</td><td>8</td><td>KEMAL HUSEN</td><td>E thiopia</td><td>02:17:37</td></tr>]
```



# PROGRAMMING WITH PYTHON

## Web Scraping

- Take a table from a webpage and convert it into a DataFrame for easier manipulation using Python.
- Get all table rows in list form first and then convert that list into a DataFrame.

```
for row in rows:
 row_td = row.find_all('td')
 print(row_td)
 type(row_td)
```

```
[<td>20</td>, <td>33</td>, <td>YOGESH KUMAR SHARMA</td>, <td>India</td>, <td>02:29:47</td>]
bs4.element.ResultSet
```



# PROGRAMMING WITH PYTHON

## Web Scraping

- To remove html tags, use BeautifulSoup.
- Use the `get_text()` method to extract the text without html tags.

```
str_cells = str(row_td)
cleantext = BeautifulSoup(str_cells, "lxml").get_text()
print(cleantext)
type(cleantext)
```

```
[20, 33, YOGESH KUMAR SHARMA, India, 02:29:47]
```

```
str
```





# PROGRAMMING WITH PYTHON

## Web Scraping

- Convert the data into a DataFrame.

```
import pandas as pd
df = pd.DataFrame(list_rows)
df.head(10)
```

	0
0	[]
1	[1, 10, BERHANE TESFAY, Eritrea, 02:11:44]
2	[2, 7, MERHAWI KESETE, Eritrea, 02:11:50]
3	[3, 13, TESFAYE DEMEKE, Ethiopia, 02:11:56]
4	[4, 4, PHILEMON RONO, Kenya, 02:12:09]
5	[5, 6, ABDI FUFA, Ethiopia, 02:12:48]
6	[6, 1, LEMI BERHANU, Ethiopia, 02:14:54]
7	[7, 26, ANISH THAPA, India, 02:17:23]
8	[8, 22, MAN SINGH, India, 02:17:37]
9	[9, 8, KEMAL HUSEN, Ethiopia, 02:17:37]





# PROGRAMMING WITH PYTHON

## Web Scraping

- Clean the DataFrame by splitting the "0" column into multiple columns at the comma position.
- Use the str.split() method

```
df1 = df[0].str.split(',', expand=True)
'''expand=True parameter ensures that the split values are '''
'''returned as separate columns in the new DataFrame df1.'''
df1.head(10)
```

	0	1	2	3	4
0	[]	None	None	None	None
1	[1	10	BERHANE TESFAY	Eritrea	02:11:44]
2	[2	7	MERHAWI KESETE	Eritrea	02:11:50]
3	[3	13	TESFAYE DEMEKE	Ethiopia	02:11:56]
4	[4	4	PHILEMON RONO	Kenya	02:12:09]
5	[5	6	ABDI FUFA	Ethiopia	02:12:48]
6	[6	1	LEMI BERHANU	Ethiopia	02:14:54]
7	[7	26	ANISH THAPA	India	02:17:23]
8	[8	22	MAN SINGH	India	02:17:37]
9	[9	8	KEMAL HUSEN	Ethiopia	02:17:37]



# PROGRAMMING WITH PYTHON

## Web Scraping

- Use the strip() method to remove the opening square bracket on column "0."

```
df1[0] = df1[0].str.strip('[')
df1.head(10)
```

	0	1	2	3	4
0	]	None	None	None	None
1	1	10	BERHANE TESFAY	Eritrea	02:11:44]
2	2	7	MERHAWI KESETE	Eritrea	02:11:50]
3	3	13	TESFAYE DEMEKE	Ethiopia	02:11:56]
4	4	4	PHILEMON RONO	Kenya	02:12:09]
5	5	6	ABDI FUFA	Ethiopia	02:12:48]
6	6	1	LEMI BERHANU	Ethiopia	02:14:54]
7	7	26	ANISH THAPA	India	02:17:23]
8	8	22	MAN SINGH	India	02:17:37]
9	9	8	KEMAL HUSEN	Ethiopia	02:17:37]



# PROGRAMMING WITH PYTHON

## Web Scraping

- The table is missing table headers.
- Use the find\_all() method to get the table headers.

```
col_labels = soup.find_all('th')
```

```
all_header = []
col_str = str(col_labels)
cleantext2 = BeautifulSoup(col_str, "lxml").get_text()
all_header.append(cleantext2)
print(all_header)
```

```
['[RANK, BIB, NAME, NATIONALITY, FINISH TIME]']
```



# PROGRAMMING WITH PYTHON

## Web Scraping

- Convert the list of headers into a pandas DataFrame

```
df2 = pd.DataFrame(all_header)
df2.head()
```

0

0 [RANK, BIB, NAME, NATIONALITY, FINISH TIME]

- Split column "0" into multiple columns at the comma position for all rows.

```
df3 = df2[0].str.split(',', expand=True)
df3.head()
```

0 1 2 3 4

0 [RANK BIB NAME NATIONALITY FINISH TIME]



# PROGRAMMING WITH PYTHON

## Web Scraping

- Concatenate the two data frames into one using the `concat()` method.

```
frames = [df3, df1]
df4 = pd.concat(frames)
df4.head(10)
```

	0	1	2	3	4
0	[RANK	BIB	NAME	NATIONALITY	FINISH TIME]
0	]	None	None	None	None
1	1	10	BERHANE TESFAY	Eritrea	02:11:44]
2	2	7	MERHAWI KESETE	Eritrea	02:11:50]
3	3	13	TESFAYE DEMEKE	Ethiopia	02:11:56]
4	4	4	PHILEMON RONO	Kenya	02:12:09]
5	5	6	ABDI FUFA	Ethiopia	02:12:48]
6	6	1	LEMI BERHANU	Ethiopia	02:14:54]
7	7	26	ANISH THAPA	India	02:17:23]
8	8	22	MAN SINGH	India	02:17:37]



# PROGRAMMING WITH PYTHON

## Web Scraping

- Assign the first row to be the table header

```
df5 = df4.rename(columns=df4.iloc[0])
df5.head()
```

	[RANK	BIB	NAME	NATIONALITY	FINISH TIME]
0	[RANK	BIB	NAME	NATIONALITY	FINISH TIME]
0	]	None	None	None	None
1	1	10	BERHANE TESFAY	Eritrea	02:11:44]
2	2	7	MERHAWI KESETE	Eritrea	02:11:50]
3	3	13	TESFAYE DEMEKE	Ethiopia	02:11:56]



# PROGRAMMING WITH PYTHON

## Web Scraping

- Get overview of the data

```
df5.info()
```

```
df5.shape
```

```
<class 'pandas.core.frame.DataFrame'>
```

```
Index: 22 entries, 0 to 20
```

```
Data columns (total 5 columns):
```

#	Column	Non-Null Count	Dtype
0	[RANK	22 non-null	object
1	BIB	21 non-null	object
2	NAME	21 non-null	object
3	NATIONALITY	21 non-null	object
4	FINISH TIME]	21 non-null	object

```
dtypes: object(5)
```

```
memory usage: 1.0+ KB
```

```
(22, 5)
```



# PROGRAMMING WITH PYTHON

## Web Scraping

- Drop all rows with any missing values.

```
df6 = df5.dropna(axis=0, how='any')
```

df6

	[RANK	BIB		NAME	NATIONALITY	FINISH TIME]
0	[RANK	BIB		NAME	NATIONALITY	FINISH TIME]
1	1	10		BERHANE TESFAY	Eritrea	02:11:44]
2	2	7		MERHAWI KESETE	Eritrea	02:11:50]
3	3	13		TESFAYE DEMEKE	Ethiopia	02:11:56]
4	4	4		PHILEMON RONO	Kenya	02:12:09]

- The table header is replicated as the first row in df5.





# PROGRAMMING WITH PYTHON

## Web Scraping

- Drop the row.

```
df7 = df6.drop(df6.index[0])
df7.head()
```

[RANK		BIB	NAME	NATIONALITY	FINISH TIME]
1	1	10	BERHANE TESFAY	Eritrea	02:11:44]
2	2	7	MERHAWI KESETE	Eritrea	02:11:50]
3	3	13	TESFAYE DEMEKE	Ethiopia	02:11:56]
4	4	4	PHILEMON RONO	Kenya	02:12:09]
5	5	6	ABDI FUFA	Ethiopia	02:12:48]



# PROGRAMMING WITH PYTHON

## Web Scraping - Analysis

- Remove the closing and opening bracket for cells.

```
print(df7.columns.tolist())
df7.columns = df7.columns.str.strip().str.replace(']', '', regex=False)
df7.columns = df7.columns.str.replace('[', '', regex=False)
print(df7.columns.tolist())
```

```
['[RANK', ' BIB', ' NAME', ' NATIONALITY', ' FINISH TIME']]
['RANK', 'BIB', 'NAME', 'NATIONALITY', 'FINISH TIME']
```



# PROGRAMMING WITH PYTHON

## Web Scraping

- Remove the closing bracket for cells in the 'FINISH TEAM' column.

```
df7['FINISH TIME'] = df7['FINISH TIME'].str.strip(']')
df7.head()
```

	RANK	BIB	NAME	NATIONALITY	FINISH TIME
1	1	10	BERHANE TEFAY	Eritrea	02:11:44
2	2	7	MERHAWI KESETE	Eritrea	02:11:50
3	3	13	TESFAYE DEMEKE	Ethiopia	02:11:56
4	4	4	PHILEMON RONO	Kenya	02:12:09
5	5	6	ABDI FUFA	Ethiopia	02:12:48



# PROGRAMMING WITH PYTHON

## Web Scraping

- Convert the FINISH TIME column to a list first for manipulation

```
time_list = df7['FINISH TIME'].tolist()
'''The FINISH TIME column is extracted from the DataFrame.'''
time_mins = []
'''An empty list is created.'''
for t in time_list:
 '''Each finish time string is processed one by one.'''
 h, m, s = t.split(':')
 '''The time string is split using colon as the separator.'''
 minutes = (int(h) * 3600 + int(m) * 60 + int(s)) / 60
 '''Hours are converted into seconds, Minutes are converted into seconds.
 Seconds are added directly. Total seconds are divided by 60.'''
 time_mins.append(minutes)
 '''Computed minutes value is added to the list.'''
print(time_mins[:10])
'''Only the first ten converted values are displayed.'''
```



# PROGRAMMING WITH PYTHON

## Web Scraping

- Convert the FINISH TIME column to a list first for manipulation

```
time_mins = [
 (int(h) * 3600 + int(m) * 60 + int(s)) / 60
 for h, m, s in (t.split(':') for t in df7['FINISH TIME'])
]
```

```
for i in time_mins:
 print(i)
```



# PROGRAMMING WITH PYTHON

## Web Scraping

- Convert the list back into a DataFrame and make a new column

```
df7['RUNNER_MINS'] = time_mins
df7.head()
```

	RANK	BIB	NAME	NATIONALITY	FINISH TIME	RUNNER_MINS
1	1	10	BERHANE TEFAY	Eritrea	02:11:44	131.733333
2	2	7	MERHAWI KESETE	Eritrea	02:11:50	131.833333
3	3	13	TESFAYE DEMEKE	Ethiopia	02:11:56	131.933333
4	4	4	PHILEMON RONO	Kenya	02:12:09	132.150000
5	5	6	ABDI FUFA	Ethiopia	02:12:48	132.800000



# PROGRAMMING WITH PYTHON

## Web Scraping

- Calculate statistics for numeric columns only in the dataframe.

```
df7['RUNNER_MINS'].describe()
```

```
count 20.000000
mean 139.757500
std 6.171218
min 131.733333
25% 134.375000
50% 139.250000
75% 145.258333
max 149.783333
Name: RUNNER_MINS, dtype: float64
```



# PROGRAMMING WITH PYTHON

## Web Scraping

- Get the smallest and largest values.

```
df7.nsmallest(5, 'RUNNER_MINS')
```

RANK	BIB	NAME	NATIONALITY	FINISH TIME	RUNNER_MINS
1	1 10	BERHANE TESFAY	Eritrea	02:11:44	131.733333
2	2 7	MERHAWI KESETE	Eritrea	02:11:50	131.833333
3	3 13	TESFAYE DEMEKE	Ethiopia	02:11:56	131.933333
4	4 4	PHILEMON RONO	Kenya	02:12:09	132.150000
5	5 6	ABDI FUFA	Ethiopia	02:12:48	132.800000

```
df7.nlargest(5, 'RUNNER_MINS')
```

RANK	BIB	NAME	NATIONALITY	FINISH TIME	RUNNER_MINS
20	20 33	YOGESH KUMAR SHARMA	India	02:29:47	149.783333
19	19 36	ANANT GAONKAR	India	02:29:14	149.233333
18	18 38	PRATHAMESH PARAMKAR	India	02:28:41	148.683333
17	17 35	LATE CHANGDEV	India	02:26:24	146.400000
16	16 37	DEV SINGH DAGAR	India	02:25:59	145.983333





# PROGRAMMING WITH PYTHON

## Web Scraping

- Get the mean based on Nationality.

```
df7.groupby('NATIONALITY')['RUNNER_MINS'].mean()
```

NATIONALITY

Eritrea 131.783333

Ethiopia 135.153333

India 143.513636

Kenya 132.150000

Netherlands 145.016667

Name: RUNNER\_MINS, dtype: float64



# PROGRAMMING WITH PYTHON

## Web Scraping

- Count how many times each nationality appears in the dataset.

```
df7['NATIONALITY'].value_counts()
```

NATIONALITY

India 11

Ethiopia 5

Eritrea 2

Kenya 1

Netherlands 1

Name: count, dtype: int64



# PROGRAMMING WITH PYTHON

## Web Scraping

- Calculate the median finish time for each nationality.

```
df7.groupby('NATIONALITY')['RUNNER_MINS'].median()
```

NATIONALITY

Eritrea 131.783333

Ethiopia 134.900000

India 142.400000

Kenya 132.150000

Netherlands 145.016667

Name: RUNNER\_MINS, dtype: float64



# PROGRAMMING WITH PYTHON

## Web Scraping

- Create a new column showing the time difference between consecutive runners.

```
df7['TIME_DIFF'] = df7['RUNNER_MINS'].diff()
```

```
df7.head()
```

	RANK	BIB	NAME	NATIONALITY	FINISH TIME	RUNNER_MINS	TIME_DIFF
1	1	10	BERHANE TESFAY	Eritrea	02:11:44	131.733333	NaN
2	2	7	MERHAWI KESETE	Eritrea	02:11:50	131.833333	0.100000
3	3	13	TESFAYE DEMEKE	Ethiopia	02:11:56	131.933333	0.100000
4	4	4	PHILEMON RONO	Kenya	02:12:09	132.150000	0.216667
5	5	6	ABDI FUFA	Ethiopia	02:12:48	132.800000	0.650000



# PROGRAMMING WITH PYTHON

## Web Scraping

- Sort the data Data Frame based on RUNNER\_MINS.

```
df7.sort_values('RUNNER_MINS')
```

	RANK	BIB	NAME	NATIONALITY	FINISH TIME	RUNNER_MINS	TIME_DIFF
1	1	10	BERHANE TESFAY	Eritrea	02:11:44	131.733333	NaN
2	2	7	MERHAWI KESETE	Eritrea	02:11:50	131.833333	0.100000
3	3	13	TESFAYE DEMEKE	Ethiopia	02:11:56	131.933333	0.100000
4	4	4	PHILEMON RONO	Kenya	02:12:09	132.150000	0.216667
5	5	6	ABDI FUFA	Ethiopia	02:12:48	132.800000	0.650000
6	6	1	LEMI BERHANU	Ethiopia	02:14:54	134.900000	2.100000
7	7	26	ANISH THAPA	India	02:17:23	137.383333	2.483333



# PROGRAMMING WITH PYTHON

## Web Scraping

- Classify runners into performance categories based on finish time.

```
df7['CATEGORY'] = pd.cut(
df7['RUNNER_MINS'],
bins=[0, 120, 140, 160, 200],
labels=['Elite', 'Strong', 'Moderate', 'Slow']
)
#Continuous numeric vaues are divided into fixed inervals.
#Time ranges are defined- 0 to 120 minutes, 120 to 140 minutes.
#140 to 160 minutes, 160 to 200 minutes.
#RUNNER_MINS ≤ 120 → Elite
#120 < RUNNER_MINS ≤ 140 → Strong
#140 < RUNNER_MINS ≤ 160 → Moderate
#160 < RUNNER_MINS ≤ 200 → Slow
#resulting category is stored as a new column
```

```
df7.head()
```

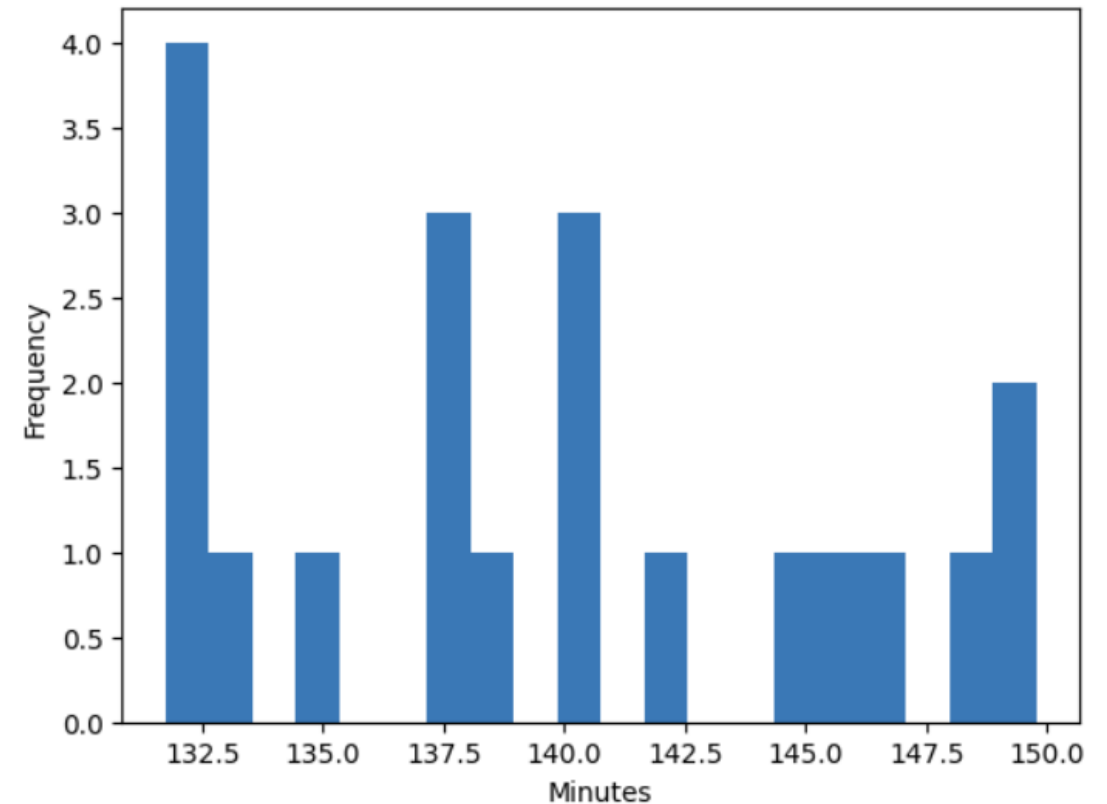


# PROGRAMMING WITH PYTHON

## Web Scraping – Data Visualization

- Create a histogram of runner finish times.

```
import matplotlib.pyplot as plt
plt.hist(df7['RUNNER_MINS'], bins=20)
plt.xlabel('Minutes')
plt.ylabel('Frequency')
plt.show()
```



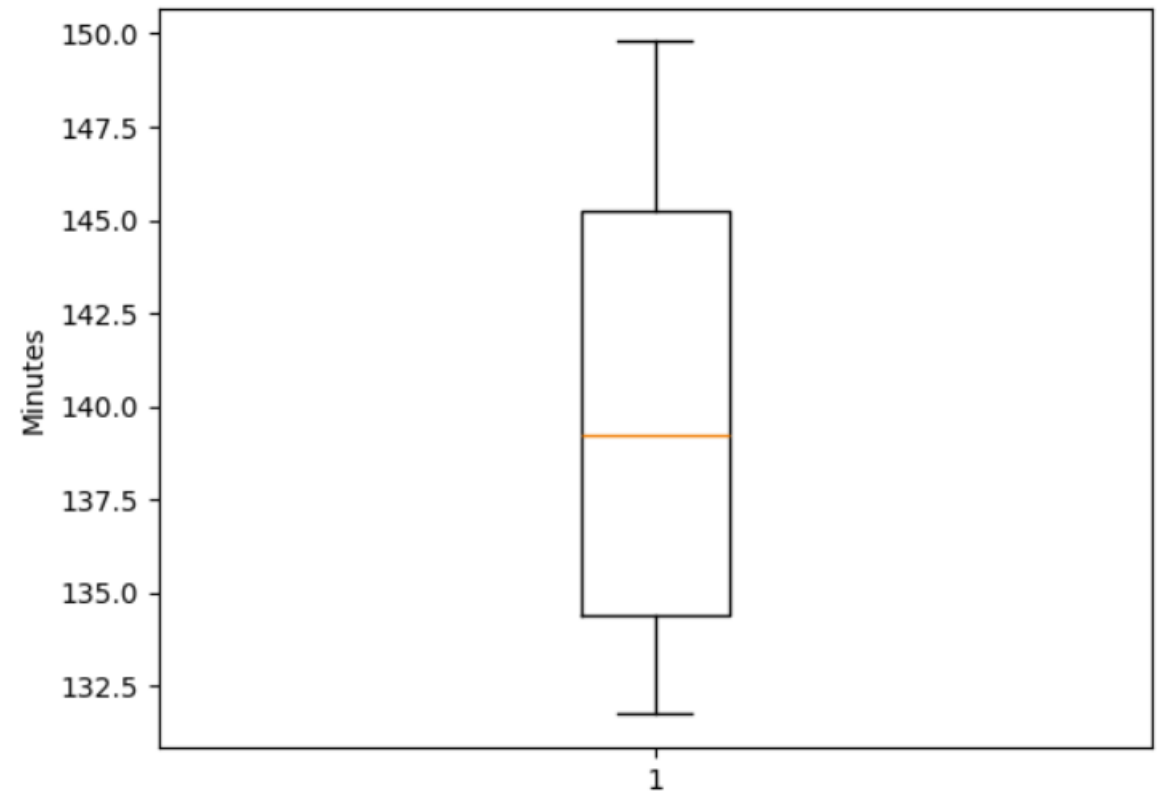


# PROGRAMMING WITH PYTHON

## Web Scraping – Data Visualization

- Create a box plot of runner finish times.

```
plt.boxplot(df7['RUNNER_MINS'])
plt.ylabel('Minutes')
plt.show()
```





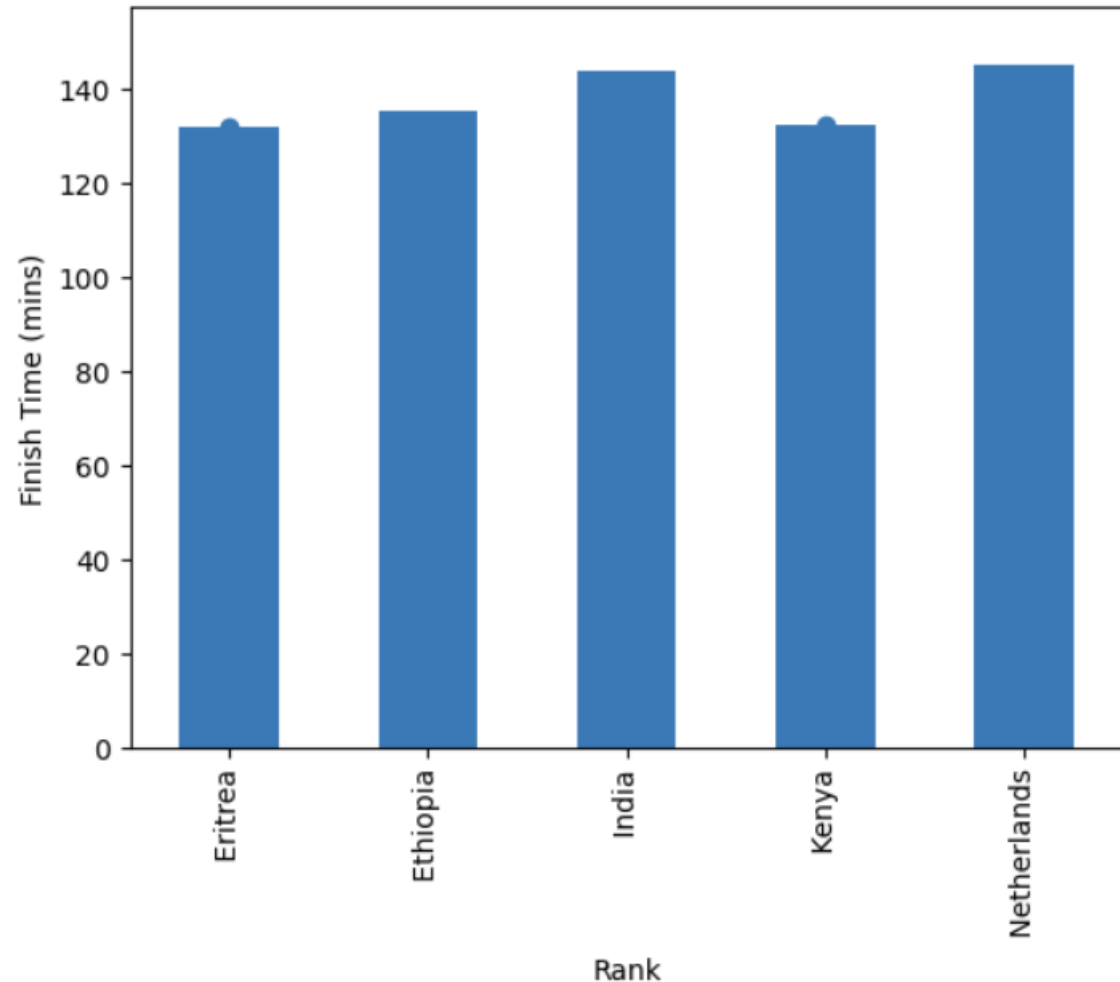


# PROGRAMMING WITH PYTHON

## Web Scraping – Data Visualiz

- Each bar represents one nationality.
- Bar height shows runner count.

```
nationality_counts = df7['NATIONALITY'].value_counts()
nationality_counts.plot(kind='bar')
df7.groupby('NATIONALITY')['RUNNER_MINS'].mean().plot(kind='bar')
plt.scatter(df7['RANK'], df7['RUNNER_MINS'])
plt.xlabel('Rank')
plt.ylabel('Finish Time (mins)')
plt.show()
```



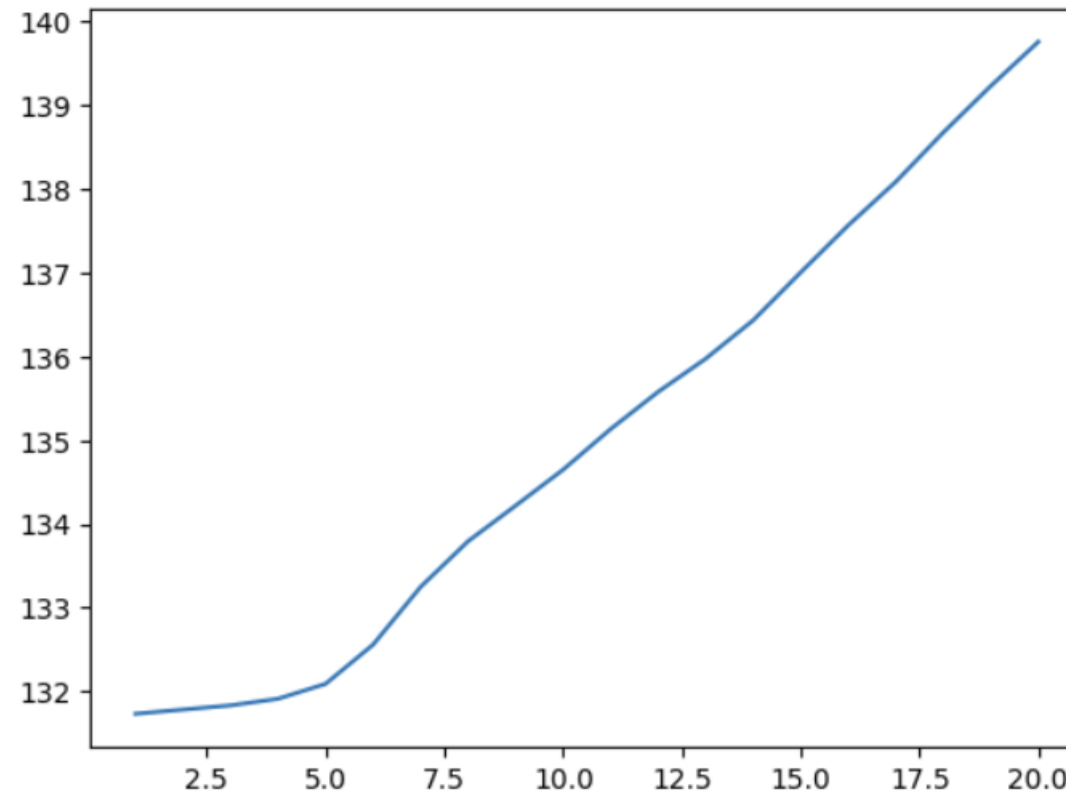


# PROGRAMMING WITH PYTHON

## Web Scraping – Data Visualization

- Plot the expanding average of runner finish times

```
plt.plot(df7['RUNNER_MINS'].expanding().mean())
plt.show()
```



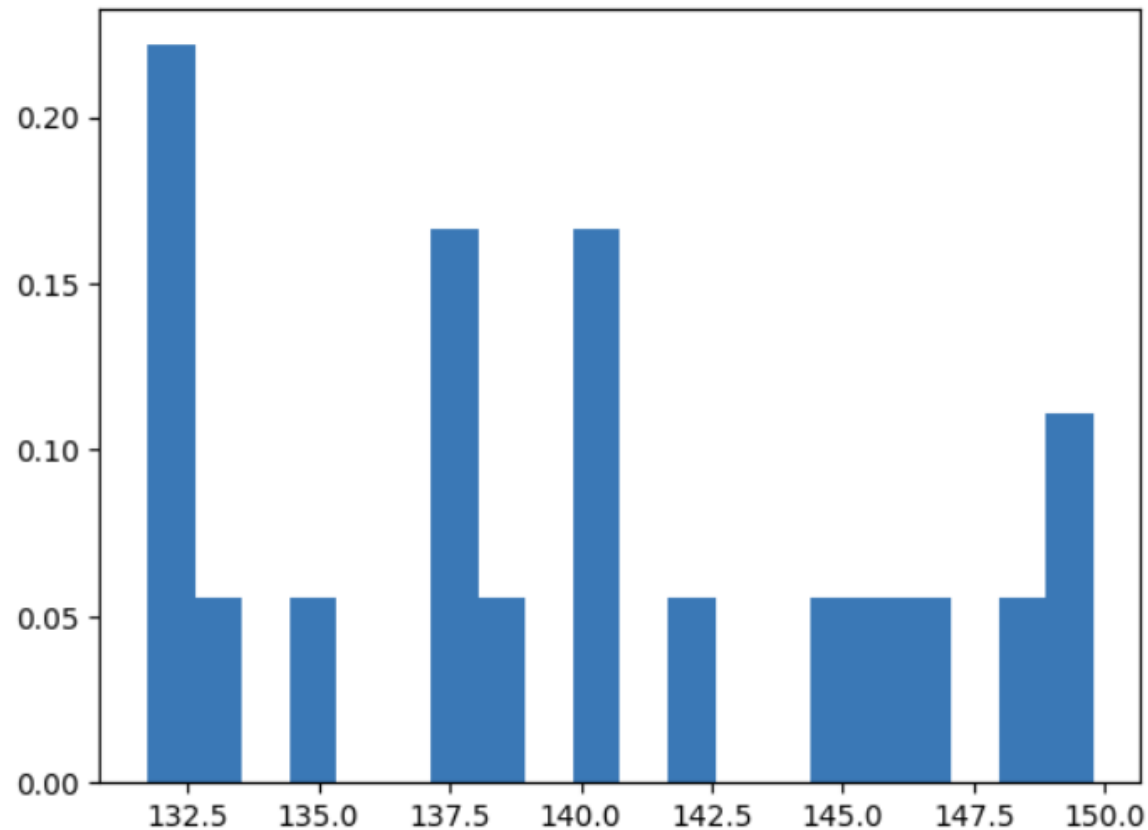


# PROGRAMMING WITH PYTHON

## Web Scraping – Data Visualization

- Draw a normalized histogram of runner finish times.

```
plt.hist(df7['RUNNER_MINS'], bins=20, density=True)
'''Values are grouped into 20 bins.'''
plt.show()
```



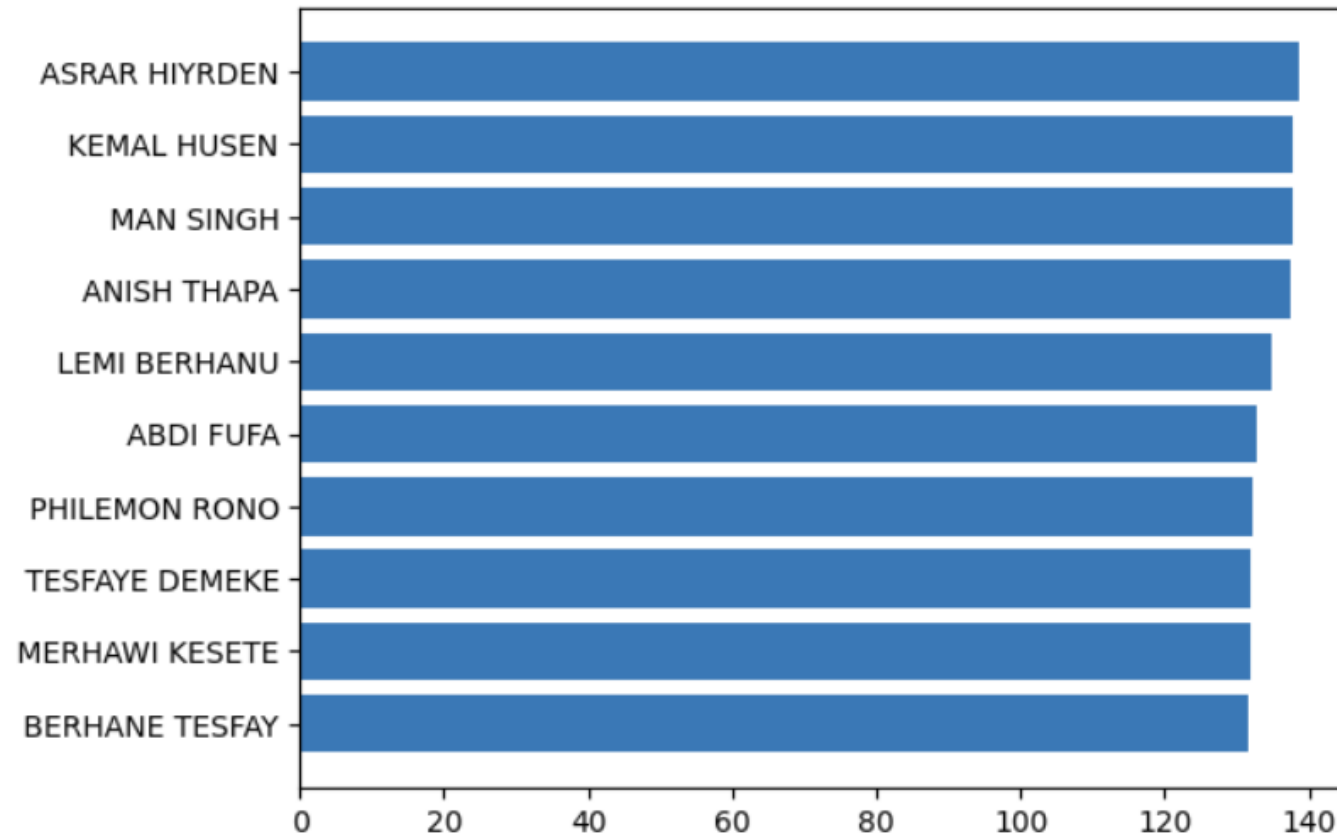


# PROGRAMMING WITH PYTHON

## Web Scraping – Data Visualization

- Highlight the fastest ten runners using a horizontal bar chart.

```
top10 = df7.nsmallest(10, 'RUNNER_MINS')
plt.barh(top10['NAME'], top10['RUNNER_MINS'])
plt.show()
```





# PROGRAMMING WITH PYTHON

## Recap

---

- Web Scraping
- BeautifulSoup object
- Converting to Data Frames
- Inspection
- Generating visualization



**THANK YOU**

---

**Lekha A**

Computer Applications